Software Requirements Specification

CompliTrace

GDPR Privacy Policy Pre-Audit Copilot

Version 1.1

Status: Draft

April 4, 2026

Contents

1 Introduction **4**
1.1 Purpose . . . 4
1.2 Scope . . . 4
1.3 Stakeholders & Intended Audience . . . 5
1.4 Definitions, Acronyms, Abbreviations . . . 5
1.5 References . . . 6

2 Product Overview **6**
2.1 Product Perspective . . . 6
2.2 Document Scope and Real-World Usage Context . . . 6
2.3 Problem Statement . . . 8
2.4 Solution Overview . . . 8
2.5 Rationale for a Dedicated System vs. General-Purpose LLM Chat . . . 8
2.6 Objectives / Goals . . . 9
2.7 Users & Roles . . . 9
2.8 Assumptions & Dependencies . . . 10
2.9 Constraints . . . 10

3 System Features (High-Level) **11**

4 AI System Requirements **13**
4.1 AI System Overview . . . 13
4.2 AI Functional Capabilities . . . 14
4.3 Agent Behavior & Workflow . . . 16
4.4 RAG Requirements . . . 17
4.5 Prompting & Context Management . . . 18
4.6 AI Non-Functional Requirements . . . 19
4.7 AI Observability & Evaluation . . . 19
4.8 AI Failure Handling . . . 20

5 Functional Requirements **21**

6 Non-Functional Requirements **24**
6.1 Performance . . . 24
6.2 Security . . . 24
6.3 Usability . . . 24
6.4 Reliability & Availability . . . 24
6.5 Scalability . . . 25
6.6 Maintainability & Observability . . . 25

7 External Interfaces **25**
7.1 User Interface . . . 25
7.2 APIs & External Services . . . 25
7.3 System Interfaces (Service-to-Service) . . . 26

8 Data Requirements **26**
8.1 Data Model Overview . . . 26
8.2 Data Validation Rules . . . 28
8.3 Data Storage & Persistence . . . 29
8.4 Data Lifecycle . . . 29

9 System Architecture & Service Model **29**
9.1 Architectural Style . . . 29
9.2 Service Overview . . . 29
9.3 Inter-Service Communication . . . 30
9.4 API Contracts & Boundaries . . . 30
9.5 Data Ownership & Isolation . . . 31
9.6 Scalability Strategy . . . 31
9.7 Fault Tolerance & Failure Scenarios . . . 31
9.8 Deployment & Infrastructure Constraints . . . 32
9.9 Observability & Monitoring . . . 32

10 Acceptance Criteria (System-Level) **32**

11 Traceability **33**

12 Risks, Constraints & Limitations **35**
12.1 Technical Risks . . . 35
12.2 AI-Specific Risks . . . 36
12.3 Operational Risks . . . 36
12.4 Known Limitations . . . 36

13 Versioning & Change Log **37**

14 Appendices **37**
14.1 Diagrams . . . 37
14.2 Sample Data . . . 39
14.3 Glossary . . . 40
14.4 TBD List . . . 40
14.5 Gold-Set Evaluation Reference . . . 41

1 Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document defines the functional requirements, non-functional requirements, AI system behavior, architecture constraints, and acceptance criteria for **CompliTrace**, a GDPR Privacy Policy Pre-Audit Copilot.

The document is intended to serve as the primary reference for the design, implementation, testing, and validation of the CompliTrace MVP. It is written in accordance with the IEEE 830 and ISO/IEC/IEEE 29148 standards for software requirements documentation.

The system is a Minimum Viable Product (MVP) scoped to a single regulatory standard (GDPR) and a single document-analysis workflow. Requirements in this document reflect that scope deliberately. Features outside the MVP boundary are explicitly excluded.

1.2 Scope

System Name

CompliTrace GDPR Privacy Policy Pre-Audit Copilot.

In-Scope

- Upload and ingestion of a single privacy or data-handling policy document (PDF).
- Automatic extraction and sectioning of document content.
- Section-by-section GDPR gap analysis driven by a bounded AI agent.
- Semantic retrieval of GDPR article chunks from a pre-indexed regulatory corpus (Qdrant vector store).
- Per-section finding generation: status, severity, gap note, remediation note, and article citations.
- Persistence of audit results in a relational database.
- Generation and download of a structured PDF gap report.
- A minimal web frontend covering: document upload, sections review, findings view, and report download.
- System observability: structured logging, Prometheus metrics, and a Grafana dashboard.
- Containerised deployment via Docker Compose.
- A basic CI/CD pipeline via GitHub Actions.

Out-of-Scope (MVP Exclusions)

- Support for regulatory standards other than GDPR (e.g., ISO 27001, CCPA, Moroccan Law 09-08).
- User authentication, user accounts, or multi-user collaboration.
- Admin panel or regulation management UI.
- Real-time streaming agent output (WebSockets).
- Document version comparison or history tracking.
- Automatic compliance scoring formulas or percentage scores.
- Notification or alerting systems.

- OCR support for scanned or image-based PDFs.
- Chatbot or conversational Q&A interface.
- Multi-tenant architecture.
- Export formats other than PDF.

1.3 Stakeholders & Intended Audience

Stakeholder	Role	Interest in this Document
Primary User	Junior compliance analyst, IT auditor, or junior consultant	Understands system capabilities and workflow
Developer	System builder (sole engineer for MVP)	Full technical reference for implementation
Technical Interviewer	Reviewer of architectural and AI design decisions	Architecture, AI, and trade-off justifications
HR / Non-Technical Reviewer	Evaluates project value and demo quality	Product overview, goals, and demo script
Future Contributor	Potential maintainer or extension developer	Requirements baseline and non-MVP backlog

1.4 Definitions, Acronyms, Abbreviations

Term	Definition
GDPR	General Data Protection Regulation (EU) 2016/679
RAG	Retrieval-Augmented Generation a technique where relevant documents are retrieved from a corpus and injected into an LLM prompt to ground the response
Agent	A software component that selects from a bounded set of actions at runtime based on the current section content and retrieved evidence
Bounded Agent	An agent whose decision space and tool access are deliberately constrained to a well-defined workflow
LLM	Large Language Model
Vector Store	A database optimised for storing and querying high-dimensional embedding vectors (Qdrant in this system)
Embedding	A numerical vector representation of text, used for semantic similarity search
Gap Finding	A documented instance where a policy section fails to meet a GDPR requirement
Citation	A reference to a specific GDPR article, paragraph, and excerpt retrieved from the corpus
Section	A logical subdivision of an uploaded policy document, identified by heading or paragraph structure
Audit	A single end-to-end execution of the gap analysis workflow on one document
SRS	Software Requirements Specification
MVP	Minimum Viable Product

Term	Definition
CI/CD	Continuous Integration / Continuous Deployment
EUR-Lex	Official EU law publication portal (source of GDPR text)
PDF	Portable Document Format
API	Application Programming Interface
REST	Representational State Transfer
UUID	Universally Unique Identifier
Processing Signal	A keyword indicating personal data processing activity; defined in Section 4.2
Evidence Sufficiency	A condition on retrieved chunks that must be met before a substantive compliance classification is assigned; defined in Section 4.3

1.5 References

- IEEE Std 830-1998 IEEE Recommended Practice for Software Requirements Specifications.
- ISO/IEC/IEEE 29148:2018 Systems and software engineering Life cycle processes Requirements engineering.
- Regulation (EU) 2016/679 (GDPR) Official text via EUR-Lex: https://eur-lex.europa.eu/eli/reg/2016/679/oj
- CompliTrace Build Plan v1.0 (internal planning document).
- FastAPI Documentation: https://fastapi.tiangolo.com
- Qdrant Documentation: https://qdrant.tech/documentation
- Sentence-Transformers: https://www.sbert.net
- Prometheus Documentation: https://prometheus.io/docs
- Grafana Documentation: https://grafana.com/docs

2 Product Overview

2.1 Product Perspective

CompliTrace is a standalone, self-contained web application with a microservice backend. It does not integrate with or extend any existing compliance management platform. It operates as an independent document analysis tool that augments the manual first-pass GDPR review process performed by compliance analysts.

The system sits at the intersection of three technology domains: document processing, retrieval-augmented generation, and structured audit reporting. It is not a workflow management tool, a control framework, or a data mapping solution. It is a narrow, citation-grounded document analysis copilot for first-pass GDPR policy review.

2.2 Document Scope and Real-World Usage Context

CompliTrace is designed to process a specific subset of privacy and compliance documents: internal company privacy or data-handling policy or procedure documents that describe how the organization handles personal data in practice. It is not intended, in the MVP, to process every document that may exist in a privacy or compliance program.

What documents exist in real organizations?

Organizations that process personal data often maintain several different kinds of documentation. In practice, these may include:

1. **Internal policy/procedure documents**
   These explain how the organization says it handles personal data internally. Examples include:
   - Employee Data Handling Policy
   - Data Retention Policy
   - Data Subject Rights Handling Procedure
   - Vendor / Processor Data Handling Procedure
   - Data Breach Response Procedure
2. **Privacy notices**
   These are notices provided to individuals whose data is collected.
3. **Controller-processor contracts / DPAs**
   These are client-specific or vendor-specific legal documents.
4. **Records of processing activities and related internal documentation.**
5. **DPIAs and risk assessments.**

Which document type does CompliTrace process?

For the MVP, CompliTrace processes **internal policy/procedure documents only**, specifically a privacy or data-handling policy document in clean PDF form.

Examples of suitable input documents include:

- Employee Data Handling Policy
- Personal Data Management Procedure
- Privacy and Data Retention Policy
- Internal Procedure for Handling Data Subject Requests

Which document types are out of scope for the MVP?

The MVP does **not** process: client-specific contracts or DPAs, public-facing website privacy notices, DPIAs, records of processing spreadsheets, vendor questionnaires, scanned or image-based PDFs, or multi-document compliance programs.

Why is CompliTrace useful if these documents are not reviewed every day?

CompliTrace is a repeatable pre-audit review tool used when a policy is first drafted, revised, when an audit is approaching, when a client requests stronger privacy documentation, or when a consultant reviews multiple client documents. Its value increases across multiple documents, versions, or consulting engagements.

Illustrative real-world example (Moroccan company)

Consider a Moroccan HR software company preparing for EU-facing client audits.

1. The company maintains an internal **Employee Data Handling Policy**.
2. Before a client due-diligence review, a junior analyst uploads the policy to CompliTrace.

3. CompliTrace splits the policy into sections such as: Data Retention, Data Subject Rights, Vendor Sharing, and Security Measures.
4. The system evaluates each section and may produce findings such as: gap in retention periods, partial coverage of data subject rights, and vague processor obligations.
5. The analyst reviews the structured report and escalates gaps to the policy owner.
6. The policy is updated and may be re-reviewed.

2.3 Problem Statement

Performing a first-pass GDPR compliance review of a company's privacy or data-handling policy is a manual, time-consuming, and error-prone process. A junior analyst must:

1. Read the policy document in full.
2. Identify which GDPR articles apply to each section.
3. Compare the section's content against those articles.
4. Document findings with citations and remediation notes.

This process typically takes several hours per document, requires familiarity with all 99 GDPR articles, and produces findings whose quality depends heavily on the analyst's experience. A single missed article or incorrect citation undermines the reliability of the entire report.

2.4 Solution Overview

CompliTrace automates the first-pass review by:

- Parsing the uploaded policy document into logical sections.
- Running a bounded AI agent that evaluates each section against a pre-indexed GDPR corpus using semantic retrieval.
- Generating per-section findings with exact article citations sourced from retrieved text, not from LLM memory.
- Producing a structured, downloadable PDF gap report suitable for analyst review.

The system is designed to support analyst review, not to provide legally conclusive compliance determinations. It produces a traceable first-pass output that the analyst validates and acts upon.

2.5 Rationale for a Dedicated System vs. General-Purpose LLM Chat

A general-purpose LLM interface can assist with policy review tasks for one-off or ad hoc analysis. CompliTrace is not justified by the claim that an LLM is incapable of performing the analysis. It is justified by turning that analysis into a **repeatable, structured, inspectable, and persistent review workflow**.

The additional value of CompliTrace over a general chat workflow is:

1. **System-enforced workflow** the analysis sequence is enforced by the application, not by prompt discipline.
2. **Persistent application records** findings, citations, and reports are stored as application-level records, not conversation content.
3. **Section-level traceability by design** each finding is linked to its originating section and to the retrieved GDPR evidence.

4. **Citation grounding and validation as a system rule** citations are validated against retrieved chunks; the constraint is enforced programmatically.
5. **Standardized output artifact** the same report structure is produced each time without depending on prompt quality.
6. **Operational visibility** metrics, logs, and lifecycle state are exposed through the observability stack.
7. **Suitability for repeated internal or consulting use** value increases when the task is performed repeatedly across documents, versions, or engagements.

The more accurate distinction is:

- **General-purpose LLM chat** suitable for flexible, ad hoc, prompt-driven assistance for one-off reviews.
- **CompliTrace** suitable when the same task should be executed as a standardized application workflow with persistent records, section-level traceability, retrieval-grounded citations, and a reusable output format across documents or over time.

2.6 Objectives / Goals

1. **Speed:** Reduce first-pass GDPR policy review time from hours to minutes.
2. **Traceability:** Every finding must cite the exact GDPR article and paragraph from which the assessment was derived.
3. **Correctness grounding:** Minimise hallucinated article citations by enforcing retrieval-grounded citation generation and output validation.
4. **Structured output:** Produce a professional, structured pre-audit gap report with per-section status, severity, and remediation notes.
5. **Observability:** All system operations are logged and metriced so that retrieval and agent behavior can be monitored and debugged.
6. **Deployability:** The entire system runs with a single `docker-compose up` command on a standard developer laptop.

2.7 Users & Roles

User Role	Description	Primary Actions
Analyst	Junior compliance analyst, IT auditor, or consultant	Upload document, review sections, trigger audit, inspect findings, download PDF report
System Administrator	Developer / deployer of the system	Run Docker Compose, monitor Grafana dashboard, re-index GDPR corpus if needed

Note: No authentication is implemented in the MVP. The system assumes a single trusted user context. Multi-user and role-based access control are out-of-scope.

2.8 Assumptions & Dependencies

Assumptions

- The uploaded document is a clean, text-extractable PDF. Scanned or image-based PDFs are not supported in the MVP.
- The GDPR corpus (EUR-Lex English text) is pre-indexed into Qdrant at system setup time and does not require runtime updates.
- An active internet connection or API key is available for the LLM inference service (Groq or Gemini Flash). A local Ollama fallback is documented but not required.
- The system is deployed in a trusted local or development environment. Production hardening (TLS, secrets management, auth) is out-of-scope.
- The demo document used during presentations is pre-authored with known intentional GDPR gaps.

Dependencies

- **Groq API / Gemini Flash API** external LLM inference endpoint.
- **EUR-Lex** source of the authoritative GDPR text corpus.
- **Qdrant** vector database for embedding storage and retrieval.
- **PostgreSQL** relational database for structured data persistence.
- **Docker & Docker Compose** container runtime for all services.
- **Sentence-Transformers** CPU-based embedding model library.
- **WeasyPrint** HTML-to-PDF rendering library.

2.9 Constraints

- **Time constraint:** The MVP must be buildable within 20 calendar days by a single developer.
- **Compute constraint:** The system must be fully operable on a laptop with 16 GB RAM and no dedicated GPU.
- **Scope constraint:** The system supports exactly one regulatory standard (GDPR) and one document type (clean text PDF) in the MVP.
- **Model constraint:** The system must not depend on frontier models (GPT-4-class). It must be feasible on Groq Llama 3.1 8B, Gemini Flash, or an equivalent local model.
- **Cost constraint:** LLM inference costs must remain negligible for demo-scale usage (10–15 sections per audit).
- **Data constraint:** The GDPR corpus must remain under 350 chunks to ensure fast embedding and retrieval on the target hardware.

3 System Features (High-Level)

Feature 1 Document Upload and Ingestion

Description: The user uploads a PDF privacy policy document. The system extracts the text, detects logical section boundaries, and persists the document and its sections for downstream analysis.

Actors: Analyst.

High-Level Flow:

1. Analyst uploads PDF via the web UI.
2. Document Ingestion Service receives the file, extracts text using `pdfplumber` or `pymupdf`.
3. Service detects section boundaries using the frozen rule set below.
4. Document metadata and sections are persisted to PostgreSQL.
5. Document status transitions to `parsed`.
6. Frontend displays the extracted sections list.

Frozen Section Detection Rule Set:
A text block is identified as a **heading** if it satisfies at least two of the following conditions:

- The block is rendered in bold or a larger font size than surrounding body text (as reported by the PDF parser).
- The block contains fewer than 15 words.
- The block appears at the start of a paragraph with no preceding body text on the same line.
- The block matches a numbered section pattern (e.g., `1.`, `1.1`, `Section 1`, `Article 1`).

Fallback rule: If fewer than two headings are detected in the entire document, the service falls back to paragraph-block splitting. In fallback mode, a new section boundary is inserted at each paragraph break where the accumulated content of the current block exceeds 200 words.

Minimum section length: A section must contain at least 50 words. Sections shorter than 50 words are merged with the immediately following section. If the merged section is still below 50 words, it is merged with the preceding section.

No-heading edge case: If the entire document contains no detectable headings and no paragraph block exceeds 200 words, the entire document is treated as a single section with title `null`.

Source span storage: If the PDF parser returns page numbers for extracted text blocks, the starting page (`page_start`) and ending page (`page_end`) of each section are stored as nullable integer fields. If page metadata is unavailable, these fields are set to `null`.

Feature 2 Sections Review

Description: Before triggering the audit, the user reviews the sections the system has identified from the uploaded document.

Actors: Analyst.

High-Level Flow:

1. Analyst navigates to the Sections Review page.
2. Frontend fetches and displays the ordered list of detected sections with titles and content previews.
3. Analyst confirms the sectioning is acceptable and triggers the audit.

Feature 3 Agentic GDPR Gap Audit

Description: The core feature of the system. The AI agent iterates through each document section, applies a pre-classification check, retrieves relevant GDPR articles, evaluates evidence sufficiency, and records structured findings.

Actors: Analyst (trigger), AI Agent (execution).

High-Level Flow:

1. Analyst triggers the audit.
2. Agent Orchestration Service loads all sections for the document.
3. For each section, the agent first applies the `not_applicable` pre-classification check (see Section 4.2).
4. If the section passes pre-classification, the agent infers the relevant GDPR topic, formulates a retrieval query, and calls the Regulatory Knowledge Service.
5. If the first retrieval result fails the frozen retry threshold, the agent reformulates and retries once.
6. The agent applies the evidence sufficiency gate. If the gate fails, the finding is immediately set to `needs_review`.
7. If the gate passes, the agent evaluates the section against the retrieved GDPR evidence using the frozen status classification rubric and assigns a status, severity, citations, gap note, and remediation note.
8. The finding is persisted to PostgreSQL.
9. After all sections are processed, the agent generates a summary.
10. Audit status transitions to `complete`.

Feature 4 Findings Review

Description: The analyst browses all findings produced by the audit and can inspect the full evidence trace for each finding.

Actors: Analyst.

High-Level Flow:

1. Analyst navigates to the Findings page.
2. Frontend displays a table of findings with status badges and severity indicators. Findings with status `not_applicable` are shown in the table but excluded from gap counts.
3. Analyst clicks a finding row.
4. A detail panel opens showing: section text, gap note, remediation note, and the retrieved GDPR article citations (article number, paragraph ref, excerpt). For `not_applicable` and `needs_review` findings, the panel shows the reason for that classification.

Feature 5 Gap Report Generation and Download

Description: The system generates a structured PDF gap report from the completed audit and makes it available for download.

Actors: Analyst.

High-Level Flow:

1. Analyst navigates to the Report page.
2. Frontend displays summary cards: counts by status and top critical gaps.
3. Analyst clicks Download PDF.

4. Agent Orchestration Service's internal report module renders an HTML template from audit data and converts it to PDF via WeasyPrint.

5. PDF is served to the browser as a file download.

Frozen Minimum Report Schema:
Every generated PDF gap report must contain the following fields in the order listed. No field may be omitted from a report that has reached `ready` status.

Field	Description
Document title	Title extracted from the uploaded document or filename
Audit timestamp	`started_at` and `completed_at` of the audit record
Audit provenance	Model provider, model name, embedding model, corpus version
Executive summary	Counts of findings by status: `compliant`, `partial`, `gap`, `needs_review`. `not_applicable` sections are listed separately and excluded from the compliance count totals.
Per-section finding block	One block per section containing: section title, section page range (if available), compliance status, severity (if applicable), gap note (if applicable), remediation note (if applicable), and all citations (article number, paragraph ref, excerpt)
Report generation metadata	Report `created_at`, report schema version

Feature 6 System Observability

Description: All services expose Prometheus metrics and emit structured JSON logs. A Grafana dashboard visualises retrieval latency, audit duration, and findings distribution.
Actors: System Administrator.
High-Level Flow:

1. All three FastAPI services expose a `/metrics` endpoint.

2. Prometheus scrapes all three endpoints on a configurable interval.

3. Grafana reads from Prometheus and renders three dashboard panels: retrieval latency histogram, audit duration gauge, findings by status bar chart.

4. Logs from all containers are emitted in JSON format to stdout and captured by Docker.

4 AI System Requirements

4.1 AI System Overview

Role of AI in CompliTrace

AI plays a central, non-decorative role in CompliTrace. The system uses a bounded AI agent as the primary analysis engine. The agent is responsible for:

- Pre-classifying sections as `not_applicable` before retrieval where appropriate.

- Inferring the regulatory topic of each substantive policy section from its content.

- Formulating dynamic retrieval queries (not template-based).
- Evaluating the quality of retrieved GDPR chunks against a frozen retry threshold.
- Applying the evidence sufficiency gate before assigning a substantive status.
- Classifying each section's compliance status using the frozen status rubric.
- Generating gap notes and remediation notes grounded in retrieved text.

Retrieval-Augmented Generation (RAG) is used to ensure that every GDPR article citation in a finding is sourced from the actual regulatory corpus, not from LLM parametric memory. This is a correctness requirement, not a design preference.

4.2 AI Functional Capabilities

Not-Applicable Pre-Classification

Before retrieval is initiated for any section, the agent applies a pre-classification check to identify administrative sections that contain no personal data processing content.

A section is immediately classified as `not_applicable` if **both** of the following conditions are true:

1. The section title (case-insensitive, stripped of punctuation) matches at least one of the following administrative patterns: `scope`, `purpose of this document`, `definitions`, `terms`, `introduction`, `overview`, `document control`, `version history`, `amendment history`, `references`, `contact us`, `contacts`.
2. The section content contains none of the following personal data processing signal keywords: `personal data`, `data subject`, `process`, `collect`, `store`, `retain`, `share`, `transfer`, `consent`, `sensitive data`, `recipient`, `controller`, `processor`.

If the section title matches an administrative pattern but the content contains at least one processing signal keyword, the section is **not** pre-classified as `not_applicable` and proceeds through the normal agent loop.

When a section is pre-classified as `not_applicable`:

- No retrieval is performed.
- The finding is persisted with status `not_applicable`, severity `null`, gap note `null`, remediation note `null`, and zero citations.
- The agent advances to the next section.

Section Topic Inference

The agent reads the raw text of each non-administrative policy section and infers the primary GDPR topic or topics that the section is likely to address (e.g., data retention, consent, data subject rights, international transfers, DPO contact).

Dynamic Retrieval Query Formulation

The agent generates a natural-language retrieval query tailored to the inferred topic and the specific content of the current section. Queries are generated dynamically at runtime from section content rather than chosen from a fixed predefined set.

Retrieval Quality Evaluation and Frozen Retry Threshold

After receiving the top-k retrieval results, the agent evaluates whether the results are semantically relevant using the following frozen threshold rule:

Retry is triggered if **either** of the following conditions is true:

1. The similarity score of the top-ranked retrieved chunk is below **0.45**; or
2. Fewer than **2** of the top-5 retrieved chunks contain at least one keyword from the inferred GDPR topic (simple keyword overlap check against the inferred topic string).

If a retry is triggered, the agent reformulates the query (broader or more specific) and calls `search_regulation` once more. A maximum of **one retry** is permitted per section. If the retry also fails the threshold, the agent proceeds to the evidence sufficiency gate with the best available results.

Evidence Sufficiency Gate

After retrieval (and any retry), the agent applies an evidence sufficiency gate before assigning a substantive compliance status.

Evidence is sufficient if **both** of the following conditions are true:

1. At least **2** of the top-5 retrieved chunks have a similarity score ≥ 0.50; and
2. At least **1** of the top-5 retrieved chunks contains at least one GDPR obligation keyword: `shall`, `must`, `required`, `obligation`, `necessary`, `appropriate`.

If evidence is **not sufficient**, the finding is immediately set to `needs_review` regardless of section content. No substantive classification (`compliant`, `partial`, or `gap`) may be assigned without passing the evidence sufficiency gate.

Compliance Status Classification and Frozen Rubric

If evidence is sufficient, the agent classifies the section using the following frozen decision rubric. The rubric must be applied in order; the first matching rule determines the status.

Status	Classification Rule
`compliant`	All core GDPR obligations inferred for the section's topic are addressed in the section text and are supported by retrieved evidence. No material gap is detectable.
`partial`	At least one core obligation is addressed in the section text and at least one core obligation is missing, vague, or only partially addressed relative to the retrieved evidence.
`gap`	A core GDPR obligation expected for the section's inferred topic is **absent** from the section text, as established by retrieved evidence. The section does not address the obligation at all, even vaguely.
`needs_review`	Applied when: (a) the evidence sufficiency gate fails; (b) the retrieved evidence is conflicting and a clear classification cannot be made; (c) the section text is too ambiguous to evaluate against retrieved evidence; or (d) the LLM output fails to parse as valid JSON.
`not_applicable`	Applied exclusively by the pre-classification check (Section 4.2). Not assigned by the LLM classification step.

Additional constraint: No section may be labeled `compliant`, `partial`, or `gap` unless the evidence sufficiency gate has been passed.

Finding Generation

For each section with a substantive status, the agent produces:

- A gap note describing what is missing or insufficient (required for `gap` and `partial`; null for all others).
- A remediation note describing what the policy should state (required for `gap` and `partial`; null for all others).
- A list of citations referencing retrieved GDPR chunks only.

Severity Assignment Rules

Severity must be assigned according to the following frozen rules with no exceptions:

Status	Severity Rule	Allowed Values
`gap`	Severity **required**. Must be non-null.	`low`, `medium`, `high`
`partial`	Severity **required**. Must be non-null.	`low`, `medium`, `high`
`compliant`	Severity **must be null**.	`null` only
`needs_review`	Severity **must be null**.	`null` only
`not_applicable`	Severity **must be null**.	`null` only

Semantic Search (RAG)

The Regulatory Knowledge Service provides semantic retrieval over the pre-indexed GDPR corpus. It accepts a natural-language query and returns the top-k most semantically similar GDPR article chunks with full citation metadata including similarity scores.

4.3 Agent Behavior & Workflow

Multi-Step Reasoning

The agent executes one reasoning cycle per document section. Each cycle includes: not-applicable pre-check, topic inference (if applicable), query formulation, retrieval, retry threshold evaluation, optional retry, evidence sufficiency gate, compliance classification using the frozen rubric, and finding generation. Within each cycle, the output of each step is a direct input to the next step.

Tool Usage

The agent has access to the following tools:

Tool	Description
`search_regulation(query, k)`	Calls the Regulatory Knowledge Service to retrieve the top-k GDPR chunks most relevant to the query. Returns chunk content, citation metadata, and similarity scores.
`get_chunk(chunk_id)`	Fetches the full content and metadata of a specific GDPR chunk by ID.

Tool	Description
`mark_finding(section_id, status, severity, citations, gap_note, remediation_note)`	Persists a finding for the current section to the database.
`next_section()`	Advances the agent loop to the next document section.

Autonomy Boundaries

The agent is a **bounded audit workflow**. Its autonomy is deliberately constrained:

- It processes sections in order; it does not reorder, skip, or group sections.
- It applies the not-applicable pre-classification check before retrieval on every section.
- It performs a maximum of one retrieval retry per section, triggered only by the frozen threshold rule.
- The evidence sufficiency gate must be evaluated on every section before a substantive status is assigned.
- It does not invoke external systems beyond the Regulatory Knowledge Service and the database.
- It does not plan across sections or revise earlier findings based on later sections.
- It does not call tools outside its defined tool set.

These constraints are intentional. They make the agent's behavior predictable, auditable, and debuggable.

4.4 RAG Requirements

Retrieval Strategy

- Top-k semantic retrieval with $k = 5$.
- Retrieval is performed over the GDPR corpus only (metadata filter: `source = GDPR`).
- Chunks are ranked by cosine similarity between the query embedding and the stored chunk embeddings.
- Similarity scores are returned with every chunk and are used by the frozen retry threshold rule and the evidence sufficiency gate.
- No reranker is applied in the MVP; baseline sentence-transformers retrieval is expected to be adequate for this bounded GDPR corpus and will be validated during early testing.

Embedding Approach

- Model: `sentence-transformers/all-MiniLM-L6-v2` (or multilingual variant see TBD-1).
- Inference: CPU only; no GPU required.
- The GDPR corpus is embedded once at setup time and stored in Qdrant.
- The same model is used at query time to embed the retrieval query.

Context Construction

- The agent's LLM prompt for each section includes:
  1. The full text of the current policy section.
  2. The top-k retrieved GDPR chunks with article number, paragraph ref, chunk ID, and content.
  3. A structured instruction specifying the required output format (status, severity, gap note, remediation note, citations).
- The agent must only cite articles present in the retrieved chunks. It must not supplement citations from parametric memory.

Frozen Citation Validation Rules

Every citation generated by the agent must pass all three of the following validation checks before it is persisted. Any citation that fails any check is rejected and replaced by a parse-failure note in the finding:

1. **Article number check:** The `article_number` field of the citation must exactly match the `article_number` field of at least one chunk in the retrieved top-k set for that section.
2. **Paragraph reference check:** If the `paragraph_ref` field is non-null, it must match the `paragraph_ref` of the corresponding retrieved chunk. If the agent cannot match the paragraph ref, the citation is stored with `paragraph_ref = null` rather than a fabricated value.
3. **Chunk ID check:** Each citation must store the `chunk_id` (Qdrant point ID) of the retrieved chunk it references. A citation without a traceable `chunk_id` must not be persisted.

These rules are enforced programmatically in the Orchestration Service after LLM output is parsed, before the finding is written to the database.

4.5 Prompting & Context Management

Prompt Structure

Each LLM inference call follows this high-level structure:

1. **System prompt:** Defines the agent's role (GDPR compliance analyst), output format (structured JSON), the frozen status rubric labels, and the grounding constraint (cite only from retrieved articles; include chunk ID in every citation).
2. **User prompt:** Contains the section text and the retrieved GDPR evidence with similarity scores and chunk IDs.
3. **Output format:** JSON with fields `status`, `severity`, `gap_note`, `remediation_note`, `citations` (list of `chunk_id` + article number + paragraph ref + excerpt).

Context Limits and Truncation

- Target context window per call: under 4,000 tokens.
- If the section text exceeds 1,500 tokens, it is truncated with a note appended.
- Each retrieved chunk is capped at 300 tokens before inclusion.
- LLM output is parsed as JSON; if parsing fails, the finding is marked `needs_review` and the raw output is logged for debugging.

4.6 AI Non-Functional Requirements

ID	Requirement	Target	Notes
AI-NFR-1	Retrieval latency	≤ 500 ms per query	Measured at the Knowledge Service
AI-NFR-2	LLM inference latency	≤ 10 s per section	Using Groq API; Gemini Flash similar
AI-NFR-3	Full audit duration	≤ 3 min for 10 sections	End-to-end, demo hardware
AI-NFR-4	Citation accuracy	100% of citations pass all three frozen validation checks	No memory-sourced citations permitted
AI-NFR-5	LLM inference cost	Negligible for demo usage	$\ll$ \$0.01 per audit at 8B model scale
AI-NFR-6	Output format compliance	$\geq$ 95% valid JSON on first parse	Remainder handled by fallback
AI-NFR-7	Rerun status stability	$\geq$ 80% status agreement across repeated runs on the gold-set document	Same model, prompt, temperature, and corpus configuration; see Acceptance Criterion 13

Safety and Guardrails

- The agent must not generate citations not present in the retrieved chunks. This is enforced via the system prompt, the frozen citation validation rules, and programmatic post-parse validation.
- The agent must not access external URLs or APIs beyond the Knowledge Service.
- Findings classified as `needs_review` are flagged for human analyst review and are not presented as definitive assessments.
- Findings classified as `not_applicable` are excluded from compliance count totals in the executive summary and report.
- The system presents all findings as a first-pass, analyst-assisted review. No output is presented as a legally conclusive compliance determination.

4.7 AI Observability & Evaluation

Logging

- Every LLM inference call is logged with: section ID, prompt token count, retrieved chunk IDs, similarity scores, raw LLM output, parsed finding, retry flag, and latency.
- Every retrieval call is logged with: query text, returned chunk IDs, similarity scores, whether the retry threshold was triggered, and latency.
- Citation validation results (pass/fail per citation) are logged per finding.
- All logs are emitted in structured JSON format to stdout.

Prometheus Metrics

- `retrieval_latency_seconds` histogram of retrieval durations.
- `retrieval_query_total` counter of total retrieval calls.
- `retrieval_retry_total` counter of retrieval retries triggered by the frozen threshold rule.
- `audit_duration_seconds` gauge of total audit run time.
- `findings_total` counter by status label (compliant, partial, gap, needs_review, not_applicable).
- `llm_inference_latency_seconds` histogram of LLM call durations.
- `citation_validation_failure_total` counter of citations rejected by validation.
- `evidence_gate_failure_total` counter of sections where the evidence sufficiency gate was not met.

Evaluation Metrics (Manual, Pre-Demo)

- Retrieval precision: manually verify that top-5 results are relevant for 10–15 test queries before final demo.
- Finding correctness: verify findings on the gold-set document match expected outcomes in Appendix 14.5.
- Citation traceability: verify that every citation in every finding maps to a real chunk in Qdrant via its stored `chunk_id`.
- Rerun stability: run the gold-set document three times with identical configuration and verify $\geq 80\%$ status agreement across runs.

4.8 AI Failure Handling

Retrieval Retry Logic

- Retry is triggered if the top-1 similarity score is below 0.45, or if fewer than 2 of top-5 chunks contain a keyword from the inferred GDPR topic.
- A maximum of one retry is permitted per section.
- If the retry result also fails the threshold, the agent proceeds to the evidence sufficiency gate with the best available results.
- If the evidence gate then also fails, the finding is set to `needs_review`.

LLM Output Fallback

- If the LLM response cannot be parsed as valid JSON, the finding is automatically set to status `needs_review`, severity `null`, with a note indicating a parse failure.
- The raw LLM output is stored in the application log for manual inspection.

Service Failure Handling

- If the Regulatory Knowledge Service is unreachable, the audit fails with status `failed` and a descriptive error is returned to the frontend.
- If the LLM API is unreachable, the audit fails with status `failed`.
- Partial audit state (completed findings before failure) is not discarded; the audit can be inspected up to the point of failure.

5 Functional Requirements

ID	Requirement (The system shall...)	Acceptance Criteria
FR-1	Accept the upload of a single PDF file via the web UI.	A PDF file up to 20 MB can be uploaded without error. The system returns a document ID and initial status `pending`.
FR-2	Extract all readable text from the uploaded PDF.	Text is extracted from all text-based PDF pages. Extraction failure sets document status to `failed` with an error message.
FR-3	Detect and split the document into logical sections using the frozen section detection rule set defined in Feature 1.	Sections are produced according to the heading-detection rule (two-condition minimum), fallback rule (paragraph blocks > 200 words), and minimum section length of 50 words. At least two sections are detected for any well-formed policy document. Each section has a title (or `null`) and a non-empty content field.
FR-4	Persist document metadata and all extracted sections to the database, including page span fields where available.	After parsing, document status is `parsed`. All sections are queryable via `GET /documents/{id}/sections`. `page_start` and `page_end` are non-null if the PDF parser returned page metadata.
FR-5	Display the extracted sections to the user before the audit is triggered.	The Sections Review page shows all detected sections with titles and content previews within 3 seconds of navigation.
FR-6	Allow the user to trigger a GDPR gap audit on a parsed document.	A `POST /audits` request with a valid document ID creates an audit record with status `pending` and returns an audit ID.
FR-7	For each document section that is not pre-classified as `not_applicable`, infer the relevant GDPR topic and formulate a retrieval query.	A unique retrieval query is generated for each substantive section; queries are not identical for sections with clearly different content.
FR-8	Retrieve the top-5 most semantically relevant GDPR chunks from the vector store for each substantive section, including similarity scores.	Each retrieval call returns exactly 5 chunks (or fewer if the corpus is smaller). Each chunk includes article number, paragraph ref, chunk ID, content, and similarity score.
FR-9	Apply the frozen retry threshold rule and retry retrieval with a reformulated query if the threshold is triggered.	Retry is triggered if top-1 score < 0.45 or fewer than 2 top-5 chunks contain an inferred topic keyword. A second retrieval call is made with a different query. No more than one retry occurs per section. The `retrieval_retry_total` metric increments on every triggered retry.

ID	Requirement (The system shall...)	Acceptance Criteria
FR-10	Apply the not-applicable pre-classification check before retrieval and classify administrative sections immediately.	Any section whose title matches a known administrative pattern and whose content contains no processing signal keywords is persisted with status `not_applicable`, null severity, null gap note, null remediation note, and zero citations. No retrieval is performed for such sections.
FR-11	Apply the evidence sufficiency gate before assigning a substantive compliance status.	If fewer than 2 top-5 chunks have score $\geq$ 0.50, or no top-5 chunk contains an obligation keyword, the finding is immediately set to `needs_review`. The `evidence_gate_failure_total` metric increments on every gate failure.
FR-12	Classify each section as `compliant`, `partial`, `gap`, `needs_review`, or `not_applicable` using the frozen classification rubric.	Every processed section has exactly one of the five valid status values. No section is left without a finding after a completed audit. No substantive status (`compliant`, `partial`, `gap`) is assigned unless the evidence sufficiency gate has been passed.
FR-13	Assign severity to each finding according to the frozen severity rules.	`gap` and `partial` findings have a non-null severity value (`low`, `medium`, or `high`). `compliant`, `needs_review`, and `not_applicable` findings have severity `null`. No exceptions are permitted.
FR-14	Generate a gap note and a remediation note for each finding where applicable.	Gap note and remediation note are non-empty strings for any finding with status `gap` or `partial`. For all other statuses, both fields are `null`.
FR-15	Validate all citations against the frozen citation validation rules before persisting.	Every citation stored in `finding_citations` passes all three checks: article number exists in retrieved top-k, paragraph ref matches retrieved metadata if non-null, and `chunk_id` references a real Qdrant point ID. Any citation failing validation is rejected and a parse-failure note is logged.
FR-16	Persist all findings and citations to the database upon completion of each section's analysis.	All findings are queryable via `GET /audits/{id}/findings` after audit completion.
FR-17	Transition the audit status to `complete` after all sections are processed.	After the final section is processed, the audit record's status field equals `complete` and `completed_at` is populated.
FR-18	Display all findings to the user in a table with status badges and severity indicators.	The Findings page renders all findings within 3 seconds. Status and severity are visually distinguished. `not_applicable` findings are shown with a distinct neutral badge and excluded from gap count totals.

ID	Requirement (The system shall...)	Acceptance Criteria
FR-19	Display the full finding detail on click, including section text, gap note, remediation note, and retrieved GDPR evidence.	Clicking a finding row opens a detail panel. For substantive findings: all five data elements are populated. For `not_applicable` and `needs_review` findings: the panel shows the classification reason.
FR-20	Generate a structured PDF gap report from a completed audit using the frozen minimum report schema.	A `POST /audits/{id}/report` on a completed audit produces a PDF containing all mandatory fields defined in the frozen report schema (Feature 5). The PDF is retrievable via `GET /audits/{id}/report/download`.
FR-21	Display an executive summary of the audit on the Report page, including counts by status.	The Report page shows summary cards for `compliant`, `partial`, `gap`, and `needs_review`. `not_applicable` sections are listed separately. All counts match the findings in the database.
FR-22	Expose Prometheus metrics from all three services.	All three services return valid Prometheus metrics format at their `/metrics` endpoints. Prometheus successfully scrapes all three targets without error.
FR-23	Display a Grafana dashboard with three panels: retrieval latency, audit duration, and findings by status.	After one complete audit, all three Grafana panels show non-zero data.
FR-24	Emit structured JSON logs from all services to stdout.	All log lines from all three services are valid JSON with at minimum: timestamp, level, service name, and message fields.
FR-25	Start the complete system stack with a single `docker-compose up` command.	Running `docker-compose up` from the repository root starts all services, and the web UI is reachable within 60 seconds.
FR-26	Persist audit provenance metadata at the time the audit record is created.	The `audits` table record for every audit includes non-null values for: `model_provider`, `model_name`, `model_temperature`, `prompt_template_version`, `embedding_model`, and `corpus_version`.
FR-27	Persist page span metadata for each section where the PDF parser provides it.	`sections.page_start` and `sections.page_end` are populated with integer values when the PDF parser returns page-level metadata. Both fields are `null` when page metadata is unavailable. No error is raised when page metadata is absent.

6 Non-Functional Requirements

6.1 Performance

ID	Requirement	Target / Rationale
NFR-P-1	Retrieval latency per query	$\leq$ 500 ms at the Knowledge Service under single-user load.
NFR-P-2	Document parsing time	$\leq$ 10 s for a 10-page text-based PDF.
NFR-P-3	Full audit duration	$\leq$ 3 minutes for a 10-section document using Groq API.
NFR-P-4	PDF report generation time	$\leq$ 15 s from trigger to file-ready.
NFR-P-5	Frontend page load time	$\leq$ 3 s for all pages under normal conditions.

6.2 Security

- The LLM API key is stored as an environment variable and never committed to version control or exposed in API responses.
- File upload is restricted to PDF MIME type. Non-PDF uploads are rejected with a `400 Bad Request` response.
- Uploaded files are stored in an isolated Docker volume, not in the web server's public directory.
- The system does not implement authentication in the MVP. It is assumed to run in a trusted local environment only.
- Inter-service communication is limited to the Docker internal network and is not exposed to external interfaces.

6.3 Usability

- The complete workflow (upload $\rightarrow$ review sections $\rightarrow$ trigger audit $\rightarrow$ view findings $\rightarrow$ download PDF) must be completable without any documentation or onboarding.
- All status indicators (audit status, finding severity, compliance status) must use clear visual differentiation (color and label, not color alone).
- Error states (parse failure, audit failure, API unavailability) must be communicated to the user with a human-readable message, not a raw error code.
- `not_applicable` findings must be visually distinguished from substantive findings throughout the UI.

6.4 Reliability & Availability

- The system must complete a full audit without failure on the pre-defined demo document in all controlled demo conditions.
- Individual service failures must not silently corrupt findings data; all failures must transition the audit or document to a `failed` status.
- The system is not required to meet production-grade availability SLAs in the MVP.

6.5 Scalability

- The MVP is designed for single-user, single-document usage. Horizontal scaling is explicitly out of scope.
- The architecture is designed to support future scaling: stateless services, shared PostgreSQL backend, Qdrant as an independent service.
- The GDPR corpus is capped at 350 chunks. If additional regulatory standards are added in a future version, the Knowledge Service's metadata filter design supports this without schema changes.

6.6 Maintainability & Observability

- All services use structured JSON logging. Log entries must include timestamp, service name, log level, and a descriptive message.
- All services expose a `/health` endpoint returning HTTP 200 when the service is operational.
- Prometheus metrics cover all critical operations: retrieval, LLM inference, audit lifecycle, document parsing, evidence gate failures, and citation validation failures.
- The Grafana dashboard provides a single-glance view of system health and audit performance.
- Code for each service is separated into distinct directories. Inter-service contracts (request/response schemas) are defined explicitly using Pydantic models.

7 External Interfaces

7.1 User Interface

The web frontend is a React + Vite + Tailwind CSS single-page application. It comprises exactly four views:

1. **Upload Page:** File input, upload button, upload progress/status indicator.
2. **Sections Review Page:** Ordered list of detected sections with title, content preview, and page range (if available). Button to trigger audit.
3. **Findings Page:** Table of all findings with status badge and severity indicator per row. `not_applicable` findings use a distinct neutral badge. Clicking a row opens a detail panel with full finding trace.
4. **Report Page:** Summary cards (counts by status, with `not_applicable` listed separately), PDF download button.

The frontend communicates with backend services exclusively via HTTP REST. Polling is used for audit status updates (no WebSockets).

7.2 APIs & External Services

External Service	Integration Point	Purpose
Groq API (primary)	Agent Orchestration Service	LLM inference for section evaluation and finding generation
Gemini Flash API (alternative)	Agent Orchestration Service	LLM inference fallback

External Service	Integration Point	Purpose
Ollama (local fallback)	Agent Orchestration Service	Offline LLM inference (documented, not required)
EUR-Lex	Offline corpus ingestion script	Source of authoritative GDPR text

7.3 System Interfaces (Service-to-Service)

Caller	Callee	Interface
React Frontend	Document Ingestion Service	`POST /documents`, `GET /documents/{id}`, `GET /documents/{id}/sections`
React Frontend	Agent Orchestration Service	`POST /audits`, `GET /audits/{id}`, `GET /audits/{id}/findings`, `POST /audits/{id}/report`, `GET /audits/{id}/report`, `GET /audits/{id}/report/download`
Agent Orchestration Service	Document Ingestion Service	`GET /documents/{id}/sections` (to load sections at audit start)
Agent Orchestration Service	Regulatory Knowledge Service	`POST /search`, `GET /chunks/{id}`
Prometheus	All three services	`GET /metrics` on each service
Grafana	Prometheus	Prometheus data source protocol

8 Data Requirements

8.1 Data Model Overview

The relational data model consists of six tables:

Table	Purpose
`documents`	Stores uploaded document metadata and parsing status
`sections`	Stores extracted sections with order, title, content, and optional page span
`audits`	Stores audit lifecycle metadata, status, timestamps, and audit provenance fields
`findings`	Stores per-section compliance findings (status, severity, notes)
`finding_citations`	Stores GDPR article citations linked to findings, including the Qdrant chunk ID
`reports`	Stores report generation status and PDF file path

Vector data (GDPR chunk embeddings) is stored in Qdrant and optionally mirrored in a `regulation_chunks` PostgreSQL table for traceability.

documents

Column	Type	Notes
id	UUID	Primary key
title	TEXT	Extracted or filename-derived
filename	TEXT	Original upload filename
status	ENUM	pending, parsed, failed
created_at	TIMESTAMP	

sections

Column	Type	Notes
id	UUID	Primary key
document_id	UUID	FK → documents
section_order	INT	Position in document
section_title	TEXT	Detected heading or null
content	TEXT	Section body text
page_start	INT	Nullable; first page of section if available from parser
page_end	INT	Nullable; last page of section if available from parser

audits

Column	Type	Notes
id	UUID	Primary key
document_id	UUID	FK → documents
status	ENUM	pending, running, complete, failed
started_at	TIMESTAMP	
completed_at	TIMESTAMP	Nullable
model_provider	TEXT	e.g., groq, gemini
model_name	TEXT	e.g., llama-3.1-8b-instant
model_temperature	FLOAT	e.g., 0.1
prompt_template_version	TEXT	e.g., v1.0
embedding_model	TEXT	e.g., all-MiniLM-L6-v2
corpus_version	TEXT	e.g., gdpr-2016-679-v1

findings

Column	Type	Notes
id	UUID	Primary key
audit_id	UUID	FK → audits
section_id	UUID	FK → sections
status	ENUM	compliant, partial, gap, needs_review, not_applicable
severity	ENUM	low, medium, high, or null
gap_note	TEXT	Nullable; required for gap and partial
remediation_note	TEXT	Nullable; required for gap and partial

finding_citations

Column	Type	Notes
id	UUID	Primary key
finding_id	UUID	FK → findings
chunk_id	TEXT	Qdrant point ID of the retrieved chunk; required
article_number	TEXT	e.g., 5
paragraph_ref	TEXT	e.g., 1(e); nullable if unresolvable
article_title	TEXT	
excerpt	TEXT	First 150 characters of chunk content

reports

Column	Type	Notes
id	UUID	Primary key
audit_id	UUID	FK → audits
status	ENUM	pending, ready, failed
pdf_path	TEXT	Local volume path
created_at	TIMESTAMP	

regulation_chunks (optional Postgres mirror)

Column	Type	Notes
id	UUID	Primary key
article_number	TEXT	
article_title	TEXT	
paragraph_ref	TEXT	
content	TEXT	
qdrant_point_id	TEXT	Qdrant point ID (matches chunk_id in finding_citations)

8.2 Data Validation Rules

- documents.status must be one of: pending, parsed, failed.
- audits.status must be one of: pending, running, complete, failed.
- findings.status must be one of: compliant, partial, gap, needs_review, not_applicable.
- findings.severity must be null when status is compliant, needs_review, or not_applicable. findings.severity must be non-null (low, medium, or high) when status is gap or partial.
- findings.gap_note and findings.remediation_note must be non-empty strings when status is gap or partial; must be null for all other statuses.
- finding_citations.chunk_id must be non-null and must reference a point ID present in the Qdrant collection.
- finding_citations.article_number must be a non-empty string matching a valid GDPR article identifier (e.g., "5", "17").

- `sections.content` must be non-empty.
- `audits.model_provider`, `audits.model_name`, `audits.model_temperature`, `audits.prompt_template_version`, `audits.embedding_model`, and `audits.corpus_version` must all be non-null at the time the audit record transitions from `pending` to `running`.
- All primary keys are UUIDs, generated server-side.
- Foreign key relationships are enforced at the database level.

8.3 Data Storage & Persistence

- All relational data is stored in PostgreSQL running in a Docker container with a named volume for persistence across restarts.
- All vector embeddings are stored in Qdrant running in a Docker container with a named volume for persistence.
- Uploaded PDF files are stored in a Docker volume mounted to the Ingestion Service.
- Generated PDF reports are stored in a Docker volume mounted to the Orchestration Service.
- No data is stored exclusively in memory; all state survives service restarts.

8.4 Data Lifecycle

- The GDPR corpus is loaded into Qdrant once at system setup via an offline ingestion script. It is not modified at runtime.
- Uploaded documents and their derived data (sections, audits, findings, citations, reports) persist indefinitely in the MVP (no TTL or deletion policy).
- There is no data purge or archival mechanism in the MVP.

9 System Architecture & Service Model

9.1 Architectural Style

CompliTrace uses a **microservice architecture** with synchronous REST communication between services. Three independent FastAPI services handle distinct bounded concerns. All services are containerised with Docker and orchestrated locally via Docker Compose.

A modular monolith would also have been an architecturally valid choice for MVP scope. The microservice split was chosen because the three concerns document ingestion, regulatory knowledge retrieval, and agent orchestration have meaningfully different responsibilities, storage backends, performance profiles, and failure modes that benefit from independent observability and deployment boundaries.

9.2 Service Overview

Service	Port	Responsibility
Document Ingestion Service	8001	Accept PDF upload; extract text; detect and split sections using frozen rule set; persist document and sections to PostgreSQL

Service	Port	Responsibility
Regulatory Knowledge Service	8002	Store GDPR embeddings in Qdrant; serve semantic retrieval queries with similarity scores; return ranked chunks with citation metadata
Agent Orchestration Service	8003	Own the audit lifecycle; run the bounded agent loop including not-applicable check, evidence gate, and citation validation; persist findings and provenance; generate PDF reports

Report generation is an internal module of the Agent Orchestration Service. It is not a separate service because its failure mode, update cadence, and deployment lifecycle are identical to orchestration.

9.3 Inter-Service Communication

All inter-service communication is synchronous HTTP/REST. No message broker or event stream is used in the MVP.

- **Frontend → Ingestion Service:** Document upload and sections retrieval.
- **Frontend → Orchestration Service:** Audit management, findings retrieval, report management, report download.
- **Orchestration Service → Ingestion Service:** Fetch sections at audit start.
- **Orchestration Service → Knowledge Service:** Semantic retrieval during the agent loop (including similarity scores).
- **Prometheus → All services:** Metrics scraping via `/metrics`.

9.4 API Contracts & Boundaries

All service APIs are defined using FastAPI with Pydantic request and response models. Contracts are auto-documented via OpenAPI at `/docs` on each service.

Service	Endpoint	Contract
Ingestion	`POST /documents`	Input: multipart PDF. Output: {`id, status, created_at`}
Ingestion	`GET /documents/{id}`	Output: {`id, title, filename, status, created_at`}
Ingestion	`GET /documents/{id}/sections`	Output: list of {`id, section_order, section_title, content, page_start, page_end`}
Knowledge	`POST /search`	Input: {`query: str, k: int`}. Output: list of {`chunk_id, article_number, paragraph_ref, article_title, content, score`}
Knowledge	`GET /chunks/{chunk_id}`	Output: full chunk object with all metadata
Orchestration	`POST /audits`	Input: {`document_id`}. Output: {`id, status`}

Service	Endpoint	Contract
Orchestration	`GET /audits/{id}`	Output: {`id, document_id, status, started_at, completed_at, model_provider, model_name, embedding_model, corpus_version`}
Orchestration	`GET /audits/{id}/findings`	Output: list of findings with nested citations including `chunk_id`
Orchestration	`POST /audits/{id}/report`	Triggers report generation. Output: {`report_id, status`}
Orchestration	`GET /audits/{id}/report`	Output: {`id, status, created_at`}
Orchestration	`GET /audits/{id}/report/download`	Output: PDF binary stream

API versioning is not implemented in the MVP. A `/v1/` prefix is recommended for any future production version.

9.5 Data Ownership & Isolation

- The **Document Ingestion Service** owns the `documents` and `sections` tables.
- The **Agent Orchestration Service** owns the `audits`, `findings`, `finding_citations`, and `reports` tables.
- The **Regulatory Knowledge Service** owns the Qdrant collection and the optional `regulation_chunks` mirror table.
- All three services share a single PostgreSQL instance in the MVP. In a production deployment, each service would own a separate database schema or instance.

9.6 Scalability Strategy

The MVP is single-user and single-instance. The architecture supports future scaling:

- Persistent state is externalized to PostgreSQL and Qdrant, allowing service restarts without losing audit data.
- Horizontal scaling of the Knowledge Service is feasible without changes to other services.
- The Orchestration Service can be made asynchronous using a task queue (e.g., Celery + Redis) to support concurrent audits in a future version.

9.7 Fault Tolerance & Failure Scenarios

Failure Scenario	Detection	Response
PDF parse failure	Ingestion Service exception	Document status → `failed`; error returned to frontend
Knowledge Service unreachable	HTTP connection error in Orchestration Service	Audit status → `failed`; error logged
LLM API unreachable	HTTP connection error in Orchestration Service	Audit status → `failed`; error logged

Failure Scenario	Detection	Response
LLM output parse failure	JSON parse error	Finding status → `needs_review`; severity → `null`; raw output logged
Retry threshold triggered, retry also fails	Frozen threshold evaluation	Proceed to evidence gate with best available results; if gate fails → `needs_review`
Evidence sufficiency gate failure	Gate evaluation logic	Finding → `needs_review`; gate failure metric incremented
Citation validation failure	Post-parse citation check	Invalid citation rejected; parse-failure note logged; finding persisted with valid citations only
Report generation failure	WeasyPrint exception	Report status → `failed`; error logged

9.8 Deployment & Infrastructure Constraints

- All services are containerised using Docker.
- The full stack is orchestrated via a single `docker-compose.yml` file.
- The system must start correctly on a machine with 16 GB RAM and no GPU.
- Docker Compose services: ingestion (8001), knowledge (8002), orchestration (8003), postgres (5432), qdrant (6333), prometheus (9090), grafana (3000).
- Service startup order is managed via Docker Compose `depends_on` and health checks.
- Environment variables (LLM API keys, database URLs) are passed via a `.env` file excluded from version control.

9.9 Observability & Monitoring

- **Prometheus:** Scrapes `/metrics` from all three services. Scrape interval: 15 seconds.
- **Grafana:** Single dashboard with three panels:
  1. Retrieval latency histogram (from Knowledge Service metrics).
  2. Audit duration gauge (from Orchestration Service metrics).
  3. Findings count by status (bar chart, from Orchestration Service metrics, including all five status values).
- **Logging:** All services emit JSON logs to Docker stdout. Log format includes: `timestamp`, `level`, `service`, `event`, and contextual fields (document_id, audit_id, section_id as applicable).
- **Health checks:** All services expose `GET /health` returning HTTP 200 when operational.

10 Acceptance Criteria (System-Level)

The following criteria define when the CompliTrace MVP is considered complete and ready for demo:

1. A PDF privacy policy document can be uploaded and parsed into at least two sections without error, using the frozen section detection rule set.

2. The extracted sections are visible in the web UI before the audit is triggered, including page span data where available.
3. A full audit can be triggered and completes without error on the pre-defined demo document.
4. Every finding produced contains: a valid status from the five-value enum, a severity value that strictly follows the frozen severity rules (non-null for `gap`/`partial`; null for all others), and notes that follow the same rules.
5. Every citation passes all three frozen citation validation checks: article number present in retrieved top-k, paragraph ref matching retrieved metadata if non-null, and a non-null `chunk_id` referencing a real Qdrant point.
6. Clicking any substantive finding in the UI shows the full detail panel including the retrieved GDPR evidence with article citation.
7. Administrative sections with no processing signals are classified as `not_applicable`, excluded from gap count totals, and shown with a distinct badge in the findings table.
8. A PDF gap report can be generated and downloaded from the UI after audit completion. The PDF contains all mandatory fields defined in the frozen minimum report schema.
9. The system starts fully with `docker-compose up` and the UI is reachable within 60 seconds.
10. All three Prometheus metric endpoints return valid data and Grafana shows live dashboard data after one completed audit.
11. The GitHub Actions pipeline passes green on the main branch.
12. The full demo workflow (upload → sections → audit → findings → report) is completable in under 6 minutes in a live presentation.
13. **Rerun stability:** Running the gold-set demo document three times in succession under identical model, prompt, temperature, and corpus configuration produces status agreement on $\geq 80\%$ of sections across all three runs.
14. **Structured output consistency:** The system produces a complete gap report containing all mandatory fields from the frozen report schema for any evaluation document without requiring per-document prompt changes or user configuration of the prompt.

11 Traceability

Requirement ID	Feature	Service	Test / Verification
FR-1	Document Upload	Ingestion	Upload valid PDF via API; verify 200 response and document ID
FR-2	Document Upload	Ingestion	Verify text extraction from demo PDF returns non-empty text
FR-3	Document Upload	Ingestion	Verify demo PDF produces sections using frozen rule set; verify ≥ 5 sections; verify min 50-word rule applied
FR-4	Document Upload	Ingestion	Query `/sections` after upload; verify all sections present; verify page_start/page_end populated or null

Requirement ID	Feature	Service	Test / Verification
FR-5	Sections Review	Frontend	Navigate to Sections page; verify list renders within 3 s
FR-6	GDPR Gap Audit	Orchestration	POST `/audits`; verify audit ID returned and status is `pending`
FR-7	GDPR Gap Audit	Orchestration	Inspect logs for section-specific queries; verify queries differ per substantive section
FR-8	GDPR Gap Audit	Knowledge	POST `/search` with test query; verify 5 chunks returned with metadata including score
FR-9	GDPR Gap Audit	Orchestration	Inject a section whose top-1 score will be below 0.45; verify retry metric increments
FR-10	GDPR Gap Audit	Orchestration	Upload demo document with an administrative section (e.g., Definitions); verify it is classified `not_applicable` with null severity and no citations
FR-11	GDPR Gap Audit	Orchestration	Verify evidence gate failure metric increments on a section with very low similarity scores; verify finding is `needs_review`
FR-12	GDPR Gap Audit	Orchestration	After audit completion, verify all sections have exactly one of five valid statuses; verify no substantive status assigned without gate pass
FR-13	GDPR Gap Audit	Orchestration	Verify all `gap/partial` findings have non-null severity; verify all `compliant/needs_review/not_applicable` have null severity
FR-14	GDPR Gap Audit	Orchestration	Verify gap_note and remediation_note non-empty for `gap/partial`; null for all others
FR-15	GDPR Gap Audit	Orchestration	Verify every citation has non-null chunk_id; verify chunk_id exists in Qdrant; verify article_number present in retrieved top-k
FR-16	GDPR Gap Audit	Orchestration	Query `/findings` after audit; verify all findings present
FR-17	GDPR Gap Audit	Orchestration	Verify audit status is `complete` and `completed_at` is set
FR-18	Findings Review	Frontend	Navigate to Findings page; verify table renders with status badges; verify `not_applicable` uses distinct badge
FR-19	Findings Review	Frontend	Click substantive finding; verify all five panel elements. Click `not_applicable` finding; verify reason shown

Requirement ID	Feature	Service	Test / Verification
FR-20	Report Generation	Orchestration	POST `/report`; verify PDF download returns valid PDF; verify all frozen report schema fields present
FR-21	Report Generation	Frontend	Verify summary cards show correct counts; `not_applicable` listed separately
FR-22	Observability	All services	Prometheus scrape all three targets without error; verify new metrics present
FR-23	Observability	Grafana	All three dashboard panels show data after one audit
FR-24	Observability	All services	Verify log lines are valid JSON with required fields
FR-25	Deployment	Infrastructure	`docker-compose up` starts all services; UI reachable in $\leq 60\,\mathrm{s}$
FR-26	Audit Provenance	Orchestration	Verify `audits` record contains non-null model_provider, model_name, temperature, prompt_template_version, embedding_model, corpus_version
FR-27	Section Spans	Ingestion	Verify page_start/page_end populated for demo PDF if parser provides page metadata; verify null if not

12 Risks, Constraints & Limitations

12.1 Technical Risks

Risk	Description	Likelihood	Mitigation
PDF section parsing quality	Arbitrary PDFs may not have clear heading structure, causing poor sectioning	Medium	Use only pre-authored demo document; document limitation explicitly
Docker Compose networking	Inter-service calls may fail due to startup ordering or hostname resolution	Medium	Use health checks and `depends_on`; test full stack early
WeasyPrint CSS compatibility	Complex HTML templates may render poorly in PDF	Low	Use minimal HTML template; test PDF output early in development
LLM API rate limits	Groq or Gemini may rate-limit rapid sequential inference calls	Low	Add configurable delay between LLM calls; use retry with backoff

12.2 AI-Specific Risks

Risk	Description	Likelihood	Mitigation
LLM classification inconsistency	The same section may receive different status on repeated runs due to LLM non-determinism	Medium	Use low temperature (0.0–0.2); validate rerun stability on gold set; seed pre-run results for demo
LLM output format failure	LLM returns non-JSON or malformed structured output	Low	JSON parse validation; fallback to `needs_review`; log raw output
Hallucinated citations from memory	LLM cites articles not in retrieved chunks despite system prompt constraint	Low	Frozen citation validation rules reject any citation whose article number is not in the retrieved top-k set
Retrieval recall gaps	Some GDPR topics may have weak semantic coverage at $k = 5$	Low	Manual retrieval validation on 15 test queries before demo; evidence gate forces `needs_review` on low-quality retrieval
not_applicable over-classification	The pre-classification check may mis-classify a substantive section as administrative	Low	Check processes processing signal keywords before classifying; test on gold-set document

12.3 Operational Risks

- **Demo environment instability:** Always seed the demo database with pre-run audit results before any presentation.
- **LLM API availability:** Document Ollama local fallback; test it before critical demos.
- **Single developer bus factor:** Maintain clear README and architecture documentation.

12.4 Known Limitations

- Only clean, text-extractable PDFs are supported. Scanned documents require OCR.
- Only the GDPR regulatory standard is supported in the MVP.
- The agent performs a maximum of one retrieval retry per section. Multi-step query planning is not implemented.
- No user authentication or access control is implemented.
- The system does not support concurrent audits or multiple simultaneous users.
- Findings are a first-pass analytical output. They do not constitute legal advice or a formal compliance determination.

- The audit trail does not support versioning or comparison of multiple audits on the same document.
- The not-applicable pre-classification uses a fixed keyword list; novel administrative section titles not in the list will not be pre-classified.
- The evidence sufficiency thresholds (0.45 retry, 0.50 gate) are set based on expected embedding model behavior and should be validated and adjusted during days 1–2 if retrieval testing reveals different score distributions.

13 Versioning & Change Log

Version	Date	Author	Description
1.0	April 3, 2026	Project Author	Initial SRS draft. MVP scope. GDPR-only. Three-service architecture.
1.1	April 4, 2026	Project Author	Precision fixes: frozen section detection rule set; frozen retry threshold (score < 0.45 or < 2 keyword matches); frozen severity rules per status; frozen citation validation rules (three checks); frozen minimum report schema. Added: not-applicable pre-classification; evidence sufficiency gate (≥ 2 chunks with score ≥ 0.50 and obligation keyword); frozen status classification rubric; audit provenance fields in `audits` table; `chunk_id` in `finding_citations`; page span fields in `sections`; rerun stability criterion (AC-13); structured output consistency criterion (AC-14); gold-set evaluation appendix. Consistency fixes: FR-13 aligned with schema; severity null rules explicit in 8.2; 7.3 interface table completed with all orchestration endpoints. FR numbering updated: FR-10 through FR-27 reflect expanded requirements.

Future versions may include: multi-standard support (Law 09-08, ISO 27001), authentication, concurrent audit support, OCR pipeline, and production deployment configuration.

14 Appendices

14.1 Diagrams

High-Level Architecture

```
[React Frontend (Vite + Tailwind)]
         |
         | HTTP
         v
++       ++
| Document Ingestion Svc    |       | Regulatory Knowledge Svc     |
| Port 8001                 |       | Port 8002                    |
```

```
| - PDF upload              |       | - GDPR corpus (Qdrant)       |
| - Text extraction         |       | - Semantic retrieval         |
| - Section detection       |       | - Citation metadata + score  |
| - PostgreSQL (docs,sects) |       | - PostgreSQL (chunks mirror) |
++       ++
         |                                     ^
         | (sections)                          | HTTP (search + scores)
         v                                     |
+--+
| Agent Orchestration Svc                              |
| Port 8003                                            |
| - Audit lifecycle + provenance                       |
| - not_applicable pre-check                           |
| - Bounded agent loop                                 |
| - Frozen retry threshold                             |
| - Evidence sufficiency gate                          |
| - Frozen classification rubric                       |
| - Citation validation (3 checks)                     |
| - LLM inference (Groq / Gemini Flash)                |
| - Findings + citations persistence                   |
| - Report generation module (WeasyPrint)              |
| - PostgreSQL (audits, findings, citations, reports) |
+--+
         |
         | metrics
         v
[Prometheus] --> [Grafana Dashboard]
```

Agent Bounded Loop (Updated)

```
START: Load all sections for document in order

FOR each section in order:

  STEP 0 | Pre-classification check
  IF section title matches administrative pattern
     AND content has no processing signal keywords:
       → Classify as not_applicable
       → Persist finding (null severity, null notes, no citations)
       → ADVANCE to next section (skip steps 1-8)

  STEP 1. Read section text
  STEP 2. Infer GDPR topic(s)
  STEP 3. Formulate retrieval query
  STEP 4. CALL search_regulation(query, k=5)  → returns chunks + scores

  STEP 5 | Frozen retry threshold check
  IF top-1 score < 0.45
     OR fewer than 2 of top-5 chunks contain inferred topic keyword:
       Reformulate query
       CALL search_regulation(new_query, k=5)  [once only]

  STEP 6 | Evidence sufficiency gate
  IF fewer than 2 top-5 chunks have score >= 0.50
```

```
    OR no top-5 chunk contains obligation keyword:
      → Classify as needs_review (null severity)
      → Persist finding
      → ADVANCE to next section (skip steps 7-8)

  STEP 7 | Frozen classification rubric + LLM evaluation
  CALL LLM with section text + retrieved chunks + scores + chunk IDs
  Parse JSON output → status / severity / gap_note / remediation_note / citations

  STEP 8 | Frozen citation validation (3 checks per citation)
  For each citation:
    Check 1: article_number in retrieved top-k? → reject if not
    Check 2: paragraph_ref matches retrieved metadata? → null if not
    Check 3: chunk_id is non-null and traceable? → reject if not
  CALL mark_finding(...)
  ADVANCE to next section

END: Generate summary → Trigger report generation
```

14.2 Sample Data

Sample Finding Object (JSON) Gap

```
{
  "id": "f3a2b1c0-...",
  "audit_id": "a1b2c3d4-...",
  "section_id": "s9e8d7f6-...",
  "section_title": "Data Retention",
  "status": "gap",
  "severity": "high",
  "gap_note": "Policy does not specify maximum retention periods per data category.",
  "remediation_note": "Define and document retention periods for each data
                       category in accordance with Article 5(1)(e) GDPR.",
  "citations": [
    {
      "chunk_id": "qdrant-point-uuid-...",
      "article_number": "5",
      "paragraph_ref": "1(e)",
      "article_title": "Principles relating to processing of personal data",
      "excerpt": "personal data shall be kept in a form which permits
                  identification of data subjects for no longer than is
                  necessary..."
    }
  ]
}
```

Sample Finding Object (JSON) Not Applicable

```
{
  "id": "f0c1d2e3-...",
  "audit_id": "a1b2c3d4-...",
  "section_id": "s1a2b3c4-...",
  "section_title": "Definitions",
  "status": "not_applicable",
  "severity": null,
```

```
  "gap_note": null,
  "remediation_note": null,
  "citations": []
}
```

Sample Retrieval Request/Response

Request:

```
POST /search
{
  "query": "data retention period storage limitation personal data",
  "k": 5
}
```

Response (first result):

```
{
  "chunk_id": "qdrant-point-uuid-...",
  "article_number": "5",
  "paragraph_ref": "1(e)",
  "article_title": "Principles relating to processing of personal data",
  "content": "Personal data shall be kept in a form which permits
              identification of data subjects for no longer than is
              necessary for the purposes for which the personal data
              are processed...",
  "score": 0.921
}
```

14.3 Glossary

See Section 1.4 (Definitions, Acronyms, Abbreviations).

14.4 TBD List

ID	Item	Notes
TBD-1	Embedding model final selection	Choose between `all-MiniLM-L6-v2` and multilingual variant after retrieval quality testing on days 1–2. Validate that expected similarity score distribution is compatible with frozen thresholds (0.45, 0.50).
TBD-2	LLM primary selection	Choose between Groq (Llama 3.1 8B) and Gemini Flash based on latency and JSON output format reliability in early agent testing
TBD-3	Section detection algorithm implementation	Finalise heading-detection heuristics using the frozen rule set against the demo document; adjust fallback threshold if needed
TBD-4	Grafana dashboard panel thresholds	Set visual alert thresholds for retrieval latency and audit duration after baseline testing

ID	Item	Notes
TBD-5	Demo document final content	Finalise the Employee Data Handling Policy with intentional gaps aligned to gold-set expectations in Appendix 14.5
TBD-6	Frozen threshold calibration	Validate that the 0.45 retry threshold and 0.50 evidence gate produce expected behavior on the gold-set document using the selected embedding model. Adjust values if score distribution differs from expectation.

14.5 Gold-Set Evaluation Reference

This appendix defines the canonical evaluation set for the CompliTrace demo document (**Employee Data Handling Policy**). It is used for:

- pre-demo finding correctness verification;
- rerun stability testing (Acceptance Criterion 13); and
- regression testing after any model, prompt, or corpus change.

The demo document must be authored to contain the sections listed below, with the content described. The expected outcomes are the ground-truth findings against which system output is evaluated.

#	Section Title	Expected Status	Expected Severity	Expected Article(s)	Key Gap / Note
1	Introduction / Document Scope	`not_applicable`	null		Administrative section; no processing signals
2	Data Retention	`gap`	High	Art. 5(1)(e)	No specific retention periods defined per data category
3	Lawful Basis for Processing	`gap`	High	Art. 6(1)	No lawful basis identified or stated for any processing activity
4	Data Subject Rights	`partial`	Medium	Art. 15–22	Rights mentioned by name but no procedure or timeline defined
5	International Data Transfers	`gap`	High	Art. 44–46	Transfers to third countries acknowledged but no transfer mechanism cited
6	Processor / Vendor Data Sharing	`partial`	Medium	Art. 28(3)	Vendors listed but processor obligations not specified
7	Security Measures	`partial`	Low	Art. 32(1)	Security mentioned in general terms; no specific technical or organisational measures described

#	Section Title	Expected Status	Expected Severity	Expected Article(s)	Key Gap / Note
8	Data Breach Notification	`gap`	High	Art. 33(1)	No reference to 72-hour supervisory authority notification requirement
9	Data Protection Officer	`gap`	Medium	Art. 37–38	DPO role not mentioned; contact point absent
10	Purpose of Processing	`compliant`	null	Art. 5(1)(b)	Processing purposes stated and limited to stated objectives

Stability criterion: On three successive runs under identical model, prompt, temperature, and corpus configuration, the system must reproduce the expected status for at least 8 of 10 sections ($\geq 80\%$) in every run. Any deviation on sections 2, 3, 5, 8, or 9 (all `gap`, High severity) should be treated as a priority investigation item.

Note on section 1: The system must classify section 1 as `not_applicable` via the pre-classification check without performing retrieval. If section 1 is classified with any substantive status, the pre-classification logic has a defect.